

← Go Back

Hardware

Portenta Vision Shield

Tutorials

Blob Detection with Portenta and OpenMV

Saving Bitmap Camera Images to the SD Card

Connect Portenta Vision Shield to TTN with LoRa® Technology

Creating a Basic Face Filter With OpenMV

Training a Custom Machine Learning Model for Portenta H7

Using the Portenta Vision Shield - Ethernet with Arduino IDE

Using the Portenta Vision Shield - Ethernet with OpenMV

Getting Started With the Portenta Vision Shield Camera

Voice Commands With The Arduino Speech Recognition Engine

Connecting to The Things Network Using OpenMV

Portenta Vision Shield User Manual

Home / Hardware / Portenta Vision Shield / **Blob Detection with Portenta and OpenMV**

Blob Detection with Portenta and OpenMV

This tutorial will show you how to use the Portenta Vision Shield to detect the presence and the position of objects in a camera image.

Author · Sebastian Romero · Last revision · 09/25/2024

ON THIS PAGE

Overview

- Goals +
- Portenta and the OpenMV IDE
- Instructions +
- Blob Detection +
- Conclusion +
- Troubleshooting +

Overview

In this tutorial you will use the Portenta Vision Shield to detect the presence and the position of objects in a camera image. To achieve this goal, you will use a technique that is often referred to as blob detection. You will write a MicroPython script and run it on the Portenta H7 with the help of the OpenMV IDE.

Goals

- How to use the OpenMV IDE to run MicroPython on Portenta
- How to use the built-in blob detection OpenMV algorithm
- How to use MicroPython to toggle the built-in LEDs

Required Hardware and Software

Portenta H7 board

Help

Arduino Portenta Vision
Shield - Ethernet

USB-C® cable

Arduino IDE 1.8.10+ or
Arduino IDE 2.0+

Portenta Bootloader Version
20+

OpenMV IDE 2.6.4+

Portenta and the OpenMV IDE

The OpenMV IDE is built for Machine Vision applications. It is meant to provide an Arduino like experience for simple computer vision tasks using a camera sensor. OpenMV comes with its own firmware that is built on MicroPython. Among other hardware, it supports the Portenta board. A statement from the creators of OpenMV on the reason why they built it:

Currently, doing anything serious involving computer vision requires a computer running an operating system running many layers of software and requiring much setup before you can get computer vision code working. This is all well and fine if you need to do many other things than just processing images, like connecting to the internet, running many different applications concurrently, etc.

But, what if, I just want to make an LED turn on if a red object appears in front of a white wall. Why do a need to build up a complex system for that? Or, what if I just want to

This is where OpenMV comes in.
[Here](#) you can read more about the OpenMV IDE.

Instructions

Configuring the Development Environment

Before you can start programming OpenMV scripts for the Portenta you need to download and install the OpenMV IDE.



IMPORTANT: Before you connect the Portenta to the OpenMV IDE make sure you update the bootloader as explained in the "Flashing the OpenMV Firmware" section!

1. Downloading the OpenMV IDE

Open the [OpenMV download](#) page in your browser and download the version that you need for your operating system. Alternatively you may use the following direct download links of the OpenMV IDE 2.6.5:

[For Windows Xp, Vista, 7, 8, 10 or Later](#)

[For OsX Snow Leopard 10.6 or Later](#)

[For Ubuntu 12.04 Lts 32-Bit or Later](#)

[For Ubuntu 12.04 Lts 64-Bit or Later](#)

[For Raspberry Pi 0, 1, 2, 3, and](#)

Follow the instructions of the installer.

2. Flashing the OpenMV Firmware

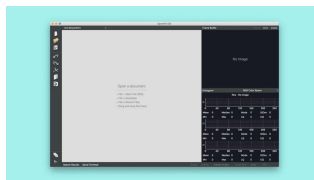
Connect the Portenta to your computer via the USB-C® cable if you haven't done so yet. Make sure you first update the bootloader to the latest version using the **STM32H747_updateBootloader** sketch in the examples menu in the Arduino IDE.

Instructions on how to update the bootloader can be found in the [Updating the Portenta Bootloader](#) tutorial.



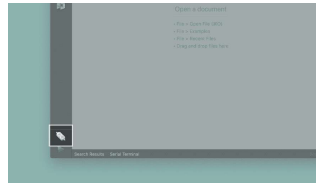
In bootloader versions 17 and older, there was a bug that could put the Portenta in a boot loop when the transmission aborted while flashing a large firmware file. This was fixed in the bootloader version 18. We strongly advise to update the bootloader before you proceed with the next step of this tutorial!

After updating the bootloader, put the Portenta in bootloader mode by double pressing the reset button on the board. The built-in green LED will start fading in and out. Now open the OpenMV IDE.



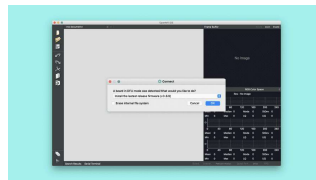
The OpenMV IDE after starting it

Click on the "connect" symbol at the bottom of the left toolbar.



Click the connect button to attach the Portenta to the OpenMV IDE

A pop-up will ask you how you would like to proceed: "DFU bootloader(s) found. What would you like to do?". Select "Reset Firmware to Release Version". This will install the latest OpenMV firmware on the Portenta. If it asks you whether it should erase the internal file system you can click "No".



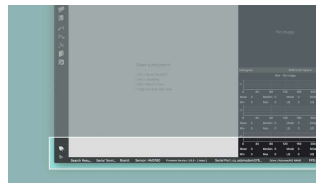
Install the latest version of the OpenMV firmware

Portenta's green LED will start flashing while the OpenMV firmware is being uploaded on the board. A terminal window will open showing you the upload progress. Wait until the green LED stops flashing and fading. You will see a message saying "DFU firmware update complete!" when the process is done.



Installing the OpenMV firmware will overwrite any existing sketches in the internal Flash of Portenta. As a result the M7 port will not be exposed in the Arduino IDE anymore. To re-flash the M7 with an Arduino firmware, you need to put the board into bootloader mode. To do so, double press the reset button on the Portenta H7 board. The built-in green LED will start fading in and out. In bootloader mode you will see the Portenta M7 port again in the Arduino IDE.

The Portenta will start flashing its blue LED when it is ready to be connected. After confirming the completion dialog, the Portenta should already be connected to the OpenMV IDE, otherwise click the "connect" button once again.



When the Portenta is successfully connected to the OpenMV IDE a green play button appears in the lower left

Blob Detection

In this section you will learn how to use the built-in blob detection algorithm to detect the location of objects in an image. That algorithm allows to detect areas in a digital image that differ in properties, such

surrounding areas. These areas are called blobs.

To do so, you need to feed an image from the camera to the algorithm. It will then analyze it and output the coordinates of the found blobs. You will visualize these coordinates directly on the image and indicate whether a blob was found by using the red and green LED.

1. Prepare the Script

Create a new script by clicking the "New File" button in the toolbar on the left side. Import the required modules:

```
1 import pyb # Import module
2 import sensor # Import the
3 import image # Import modu
4 import time # Import modu
```

A module in Python® is a confined bundle of functionality. By importing it into the script, it gets made available.

2. Preparing the Sensor

In order to take a snapshot with the camera, it has to be configured in the script.

```
1 sensor.reset() # Resets th
2 sensor.set_pixformat(sensor
3 sensor.set_framesize(sensor
4 sensor.skip_frames(time =
```

The most relevant functions in this snippet are `set_pixformat` and `set_framesize`. The camera that comes with the Portenta Vision Shield only supports grey scale images. Therefore, you need to set it via the `sensor.GRAYSCALE` parameter.

The resolution of the camera needs to be set to a supported format both by the sensor and the algorithm. Algorithms which use a neural network are usually trained on a specific image resolution, to make them sensitive to the provided image snapshot resolution. The Portenta Vision Shield supports `QVGA`, which you will use in this tutorial.

3. Detecting Blobs

In order to feed the blob detection algorithm with an image, you have to take a snapshot from the camera or load the image from memory (e.g. SD card or internal Flash). In this case, you will take a snapshot using the `snapshot()` function. The resulting image needs then to be fed to the algorithm using the `find_blobs` function. You will notice that a list of tuples gets passed to the algorithm. In this list, you can specify the grey scale values (brightness) that are mostly contained in the object that you would like to track. For instance, if you wanted to detect white objects on a black background, the resulting range of brightness would be very narrow (e.g. from 200 - 255). Remember that 255 denotes the maximum brightness / white and 0 corresponds to the minimum brightness / black. If you are

objects, you can set the threshold range for example to (100, 255).

```
1 thresholds = (100, 255) #
2 img = sensor.snapshot() #
3
4 # Find blobs with a minima
5 # Overlapping blobs will n
6 blobs = img.find_blobs([th
```

Once the blobs are detected, you may be interested to see where they have been found in the images. This can be done by drawing directly on the camera image.

```
1 # Draw blobs
2 for blob in blobs:
3     # Draw a rectangle whe
4     img.draw_rectangle(blob
5     # Draw a cross in the
6     img.draw_cross(blob.cx
```

The result of that will be visible in the Frame Buffer preview panel on the right side of the OpenMV IDE.

4. Toggling LEDs

What if you want some visual feedback from the blob detection without any computer connected to your Portenta? You could use for example the built-in LEDs to indicate whether or not a blob was found in the camera image. Let's initialize the red and the green LEDs with the following code:

```
1 ledRed = pyb.LED(1) # Init
2 ledGreen = pyb.LED(2) # In
```

And then add the logic that will turn on the appropriate LED if a blob is present. This part of the code will be added after the "Draw Blobs" logic.

```
1 # Turn the green LED on if
2 if len(blobs) > 0:
3     ledGreen.on()
4     ledRed.off()
5 else:
6 # Turn the red LED on if n
7     ledGreen.off()
8     ledRed.on()
```

In this example, the green LED will light up when there is at least one blob found in the image. The red LED will light up if no blob could be found.

5. Uploading the Script

Let's program the Portenta with the complete script and test if the algorithm works. Copy the following script and paste it into the new script file that you created.

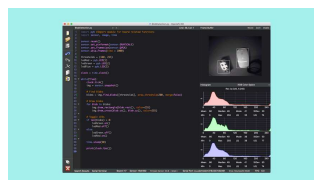
```

1 import pyb # Import module
2 import sensor # Import module
3 import image # Import module
4 import time # Import module
5
6 sensor.reset() # Resets sensor
7 sensor.set_pixformat(sensor.RGB565) # Sets pixel format to RGB565
8 sensor.set_framesize(sensor.QVGA) # Sets frame size to QVGA
9 sensor.skip_frames(time=200000) # Skip a few frames to warm up
10
11 thresholds = (100, 255) # Thresholds for blob detection
12 ledRed = pyb.LED(1) # Red LED
13 ledGreen = pyb.LED(2) # Green LED
14
15 clock = time.clock() # Create a clock object
16
17 while(True):
18     clock.tick() # Advance the clock
19     img = sensor.snapshot() # Take a snapshot of the camera
20
21     # Find blobs with a threshold
22     # Overlapping blobs
23     blobs = img.find_blobs(thresholds)
24
25     # Draw blobs
26     for blob in blobs:
27         # Draw a rectangle around the blob
28         img.draw_rectangle(blob.rect())

```

Click on the "Play" button at the bottom of the left toolbar. Place some objects on your desk and check if the Portenta can detect them.

The MicroPython script does not get compiled and linked into an actual firmware. Instead  it gets copied to the internal Flash of the Portenta where it gets compiled and executed on the fly.



An example of a blob

Conclusion

In this tutorial you learned how to use the OpenMV IDE to develop MicroPython scripts that can run on the Portenta board. You also learned how to configure the camera of the Portenta Vision Shield for machine vision applications in OpenMV. Last but not least, you learned how to interact with the built-in LEDs in MicroPython on the OpenMV firmware.

Next Steps

Familiarize yourself with the OpenMV IDE. There are many other features that have not been mentioned in this tutorial (e.g. the Serial Terminal).

Try out other machine vision examples that come with the OpenMV IDE (e.g. Face Detection). You can find them in the "Examples" menu.

Troubleshooting

OpenMV Firmware Flashing Issues

If the upload of the OpenMV firmware fails during the download, put the board back in bootloader mode and try again. Give it a few trials until the firmware gets successfully uploaded.

If the upload of the OpenMV firmware fails without even starting, try uploading the latest firmware using the "Load Specific Firmware File" option

[repository](#). Look for a file called **firmware.bin** in the PORTENTA folder.

If you experience issues putting the board into bootloader mode, make sure you first updated the bootloader to the latest version using the **STM32H747_updateBootloader** sketch from the examples menu in the Arduino IDE.

If the camera cannot get recognized by the OpenMV IDE or if you see a "No OpenMV Cams found!" message, press the reset button on the Portenta once and wait until you see the blue LED flashing. Then try again connecting to the board.

If you see a "OSError: Reset Failed" message, reset the board by pressing the reset button. Wait until you see the blue LED flashing, connect the board to the OpenMV IDE and try running the script again.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

